Quantum Optimisation Circuit Builder

Muntaser Syed, Florida Institute of Technology · DeepStation IBM Quantum: Qiskit Fall Fest Hackathon

The problem

Combinatorial optimisation problems in QUBO form (MaxCut on a weighted graph converts exactly) sit inside scheduling, routing, and portfolio construction. When a practitioner takes one to a quantum algorithm today, the algorithm and parameter choices are guesswork, and the quantum result is reported on its own, with nothing certifying how good it actually is.

What the tool does

Describe the problem and the builder does the rest. It extracts the features the algorithm choice turns on (problem size, coupling density, hardware qubit budget), recommends an algorithm and parameters with the rationale printed next to the choice, runs an exact classical solve that certifies the optimum for small instances (Gray-code enumeration, n up to 24), refuses the problem when it exceeds the stated budget, and builds the QAOA circuit via the problem's Ising form. The statevector run is then scored against the certified optimum, never reported on its own. A dependency-free web port runs the entire pipeline in the browser in exact numerical parity with the Python pipeline.

Verified demo record (6-node MaxCut, run Nov 10, 2026)

Recommendation	exact + QAOA, p = 3 (coupling density 0.53, high band)
Certified optimum	-8.000 at x = [0 1 0 0 1 0], by exact solve
QAOA circuit	6 qubits, depth 19, 6 parameters
Statevector scan	best expectation -7.308 against the -8.000 optimum (200-angle scan)
Web demo	same certified optimum; parity with the Python pipeline enforced by automated tests in the repo

Why quantum, stated honestly

QAOA is a candidate heuristic for large instances where exact solves stop scaling, but there is no guaranteed speedup and today's hardware is noisy and qubit-limited. That is exactly why this tool certifies: every quantum number it reports is anchored to a proven classical optimum at the sizes where proof is affordable, and the recommendation layer says plainly when an exact classical solve is the right answer instead. All results on this page are from a statevector simulator; no hardware run is claimed.

Links

Live demo: https://qopt-builder.vercel.app/
Code: https://github.com/jemsbhai/qopt-builder
Demo video: https://youtu.be/M5k3179ypZs